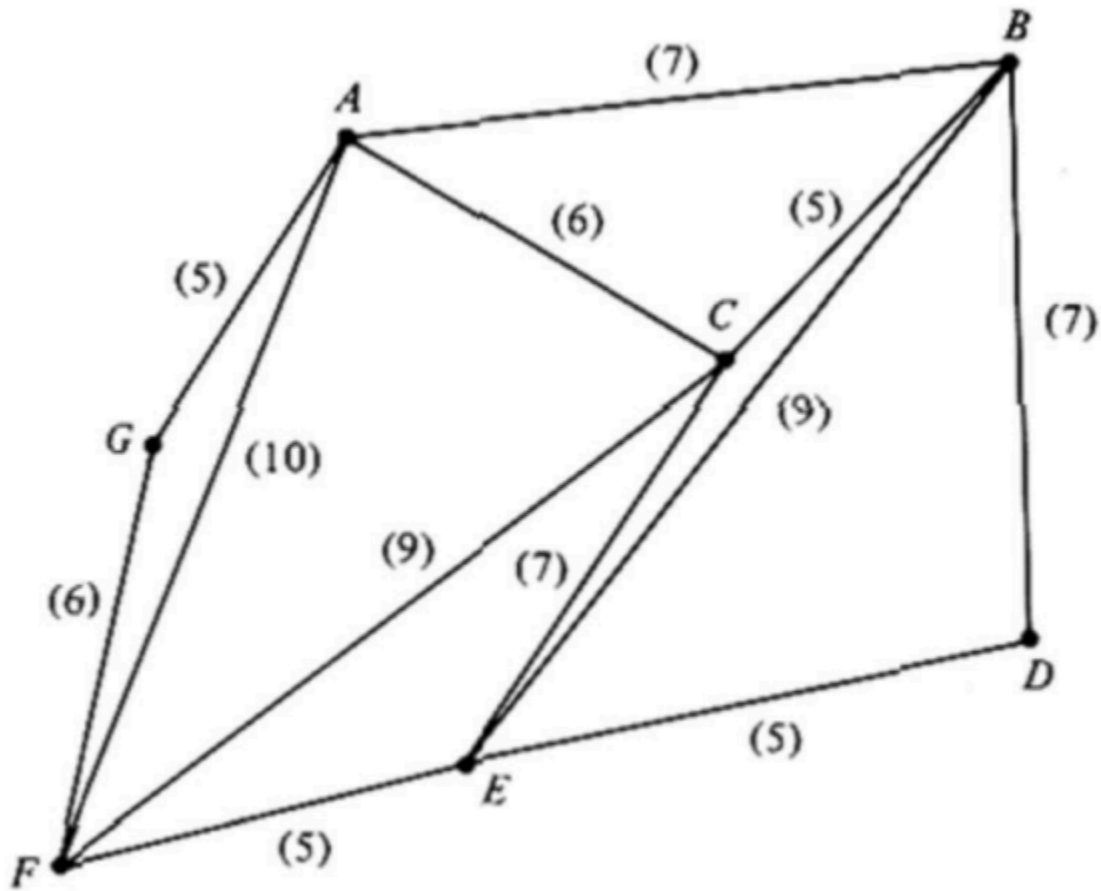


Graph Theory Group Assignment 2 Report



GROUP 8

NRP	Name
5025251260	Aqilah Ibrahim
5025251161	Rizqi Arya Kuskhilbyano
5025251009	Athar Rozy Rasyidan
5025251017	Wafi Fawwaz Sutisna

1. Prim's Algorithm

Given a weighted, connected, undirected graph, find a spanning tree that connects all vertices with the minimum possible total edge weight (the Minimum Spanning Tree, MST). This mirrors the DPTSI network-cabling scenario from the lecture: connect every lab room at the least total cable cost.

Step	U	Edge(u,v)	V - U	Graf
0	{ }	-	{A,B,C,D,E,F,G}	
1	{A}	A-B=7 A-C=6 A-G=5 ✓ A-F=10	{B,C,D,E,F}	
2	{A,G}	A-B=7 A-C=6 A-F=10 G-F=6 ✓	{B,C,D,E}	
3	{A,G,F}	A-B=7 A-C=6 A-F=10 CYCLES F-C=8 F-E=5 ✓	{B,C,D}	
4	{A,G,F,E}	A-B=7 A-C=6 A-F=10 CYCLES F-C=8 E-B=8 E-C=7 E-D=5 ✓	{B,C}	
5	{A,G,F,E,D}	A-C=6 ✓ A-B=7 A-F=10 CYCLES F-C=8 E-B=8 E-C=7 D-B=7	{B}	
6	{A,G,F,E,D,C}	A-B=7 A-F=10 CYCLES F-C=8 CYCLES E-B=8 E-C=7 CYCLES D-B=7 C-B=5 ✓	{ }	

2. Kruskal's Algorithm

All edges are sorted by weight ascending, then added one by one as long as they connect two different components, tracked with a Union-Find (DSU) structure with path compression and union by rank.

Edges selected in ascending order: A-G(5), B-C(5), D-E(5), E-F(5), A-C(6), F-G(6). Total weight = 32, matching Prim's result exactly (as MST theory guarantees).

Network Graph $G = (V, E)$
 $|V| = 7, |E| = 12$

Step	Disjoint Sets (Forest)	Edge (u, v)	Status	Graf
0	{A}, {B}, {C}, {D}, {E}, {F}, {G}	-	Initial State	
1	{A, G}, {B}, {C}, {D}, {E}, {F}	(A-G) = 5	Added ✓	
2	{A, G}, {B, C}, {D}, {E}, {F}	(B-C) = 5	Added ✓	
3	{A, G}, {B, C}, {D, E}, {F}	(D-E) = 5	Added ✓	
4	{A, G}, {B, C}, {D, E, F}	(E-F) = 5	Added ✓	
5	{A, B, C, G}, {D, E, F}	(A-C) = 6	Added ✓	
6	{A, B, C, D, E, F, G}	(F-G) = 6	Added ✓ (MST Complete)	

(graph pictures on the right courtesy of gemini)

3. Borůvka's Algorithm

The algorithm starts with each vertex as a separate component and repeatedly finds the cheapest edge connecting each component to another component. These edges are added to the MST, causing the components to merge. This process continues until all vertices are connected, resulting in a Minimum Spanning Tree with a total weight of 32

Sorted order: A-G(5), B-C(5), D-E(5), E-F(5), A-C(6), F-G(6), A-B(7), B-D(7), C-E(7), B-E(9), C-F(9), A-F(10).

Borůvka's algorithm first selects A-G, B-C, D-E, and E-F as the cheapest edges for their respective components. In the next round, A-C and F-G are selected to merge the remaining components.

Total weight = 32, matching Prim's result exactly (as MST theory guarantees)

Round 1

Component	Cheapest Edge	Weight
{A}	A-G	5
{B}	B-C	5
{C}	B-C	5
{D}	D-E	5
{E}	D-E / E-F	5
{F}	E-F	5
{G}	A-G	5

Chosen Edge:

- {A,G}
- {B,C}
- {D,E,F}

4 Edge in Round 1:

A-G, B-C, D-E, E-F

Round 2

Component:

A-G, B-C, D-E, E-F

Component	Cheapest Edge	Weight
{A,G}	A-C	6
{B,C}	A-C	6
{D,E,F}	F-G	6

4. Failure Simulation - Kruskal Algorithm

Edge Failure: F–G

Assume that the edge F–G (6) fails and cannot be traversed.

The original Minimum Spanning Tree is:

A–G(5), B–C(5), D–E(5), E–F(5), A–C(6), F–G(6)

Original total weight:

$$5 + 5 + 5 + 5 + 6 + 6 = 32$$

When F–G fails, the MST is split into two components:

- Component 1: {A, B, C, G}
- Component 2: {D, E, F}

The algorithm must find the cheapest available edge that reconnects these two components.

The possible replacement edges are:

Edge	Weight
B–D	7
C–E	7
B–E	9
C–F	9
A–F	10

The cheapest replacement is either B–D(7) or C–E(7). We choose B–D(7).

Therefore, the new MST becomes:

A–G(5), B–C(5), D–E(5), E–F(5), A–C(6), B–D(7)

New MST weight

$$5 + 5 + 5 + 5 + 6 + 7 = 33$$

So, after the failure:

Original MST weight = 32

New MST weight = 33

How the algorithm adapts

When an MST edge fails, the algorithm removes the failed edge and checks whether the graph is still connected. If the graph becomes disconnected, it searches for the minimum-weight

available edge connecting the resulting components. The failed edge F–G is excluded from further consideration, and B–D(7) is selected as its replacement.

Step	Edge	Weight	Status	Components
0	F–G	6	Failed	{A,B,C,G}, {D,E,F}
1	B–D	7	Replacement / Added	{A,B,C,D,E,F,G}

Final MST after failure:

{A-G, B-C, D-E, E-F, A-C, B-D}

Total weight: 33

5. AI Disclosure:

AI tools were used to assist in understanding the assignment, explaining Borůvka's Algorithm, checking the calculation of the Minimum Spanning Tree (MST), and improving the clarity of the documentation. The implementation, testing, and final decisions were reviewed and adjusted by the authors.